

Phase 5

Cache

EEL 4768 Computer Architecture
Fall 2026

Learning Objectives

By the end of this phase you should be able to:

- Design a set associative data and instruction cache in Verilog.
- Implement LRU and PLRU replacement policies, a write-back/write-through policy, and next line prefetching.
- Integrate a cache into a pipelined CPU

Note

You must write your own Verilog test benches to verify your designs. An example test trace will be provided, as well as a skeleton in `phase_5/skeletons/`. You also should reference the RV32I reference card from webcourses for the instruction formats, opcodes, and immediate values.

1 Submission

- **One submission per group.** You will manually submit your work to Webcourses as a zip file. Students in multiple groups, or not in a group by the time of submission, will receive **no credit** for the submission.
- **Files to submit:**

```
hart.v
alu.v
imm.v
rf.v
decoder.v
cache.v
additional verilog files (if any)
```

One module per file, and name the module the same as the filename. The testbenches instantiate modules named `hart`, `alu`, `imm`, `rf`, `decoder` and `cache.v` by name.

2 Grading Rubric

Category	Points
Base configuration	3 pts
Direct mapped	1 pts
Set associative	1 pts
PLRU	1 pts
Next line prefetching	1 pts
Total	7 pts

Section 4.5.4 lists the base configuration and what is required to earn the points for each category.

3 Verilog Coding Rules

Your submission is internally checked against a synthesizable subset. The rules listed below will result in a 0 for the specific file.

Rule	What it means for you
<code>*, /, %, **</code> only in constant expressions	Allowed in <code>parameter</code> and <code>localparam</code> definitions, in port and wire widths such as <code>[BLOCK.SIZE*8-1:0]</code> , and in generate indices. Not allowed on signals. The result must be computable at compile time.
No <code>===</code> , <code>!==</code> , <code>==?</code> , <code>!=?</code> Shifts only by a literal	Not synthesizable. Use <code>==</code> and <code>!=</code> . <code>x << 3</code> is fine; <code>x << i_op2[4:0]</code> and <code>x << SOME.PARAMETER</code> are both <i>rejected</i> . When the shift amount is a parameter, use a part-select or a concatenation instead.
No <code>if/else</code> in combinational logic	<code>if/else</code> is only legal inside a clocked <code>always @(posedge ...)</code> block. For combinational logic use a continuous <code>assign</code> with the <code>?:</code> operator, or a <code>case</code> statement inside <code>always @(*)</code> . A generate-level <code>if</code> on a parameter is fine.
No procedural loops	<code>for</code> , <code>while</code> , <code>repeat</code> and <code>do/while</code> statements are rejected. To build something repetitive, use a <code>generate</code> loop, or just write the instances out.
No delays or waits	<code>#5</code> , <code>wait</code> , <code>disable</code> , etc, are simulation constructs.
No system tasks	<code>\$display</code> , <code>\$finish</code> , <code>\$stop</code> , <code>\$system</code> , <code>\$readmemh</code> / <code>\$readmemb</code> and all file I/O (<code>\$fopen</code> , <code>\$fwrite</code> , ...) are not allowed in your submission. Only constant functions, i.e., <code>\$clog2</code> , are allowed.
No <code>'include</code> , <code>'ifdef</code> , <code>'ifndef</code> , <code>'elsif</code>	Not needed. The only conditional directive allowed is the skeleton's <code>'ifdef RISC_V_FORMAL</code> . This check happens if commented out.
Single clock, synchronous reset	<code>always @(posedge i_clk)</code> with <code>if (i_rst) ... else ...</code> inside it. No second signal in the sensitivity list. This applies to every pipeline register you add, not just the ones Phase 4 already had.

4 CPU cache integration

In phase 4 you restructured your datapath into a 5 stage pipelined processor where each stage of the pipeline works on a different instruction during the same cycle. For this, we assumed instant memory access, which is not realistic. In this phase, you will design both a data and instruction cache that interfaces with the processor and main memory to increase memory access speeds.

4.1 Cache

In a processor, a cache acts as an intermediary between the CPU and main memory. It lives on-chip with the processor logic, allowing for fast memory transactions. The cache stores a subset of the data in main memory. When the CPU requests data, it first checks if that data exists in the cache. If it does, it is considered a **cache hit** and the data is returned to the CPU. If the data is not in the cache, it is considered a **cache miss** and the cache must fetch the data from main memory. When a cache is full and a miss occurs, it must evict a block of data to make room for the new data, defined by the cache's *replacement policy*.

4.2 Cache organization

In a CPU, there are multiple levels to the memory hierarchy which defines how data is packed, stored, and accessed. A common phrase in computer architecture is that memory is *byte addressable*, but there is more structure above this to group memory into larger chunks.

4.2.1 Blocks

A block is a contiguous chunk of memory that is the smallest unit of data transfer. A block will consist of a number of bytes that defines its size. For example, a common block size is *64 bytes*, which means that each block will be comprised of 64 8-bit values, for a total of 512 bits or, surprisingly, 64 bytes.

4.2.2 Cache line

A cache line is a single block of memory that is stored in the cache. Each line contains a tag, data, and possibly other parameters used to process the block in the cache. One of the most common ones is a **valid bit**, which indicates whether the line is currently storing valid data. Another common parameter is a **dirty bit**, which indicates whether the line has been modified since it was loaded from main memory. A dirty bit is important, as it tells you if you need to write data back to main memory upon eviction.

4.3 Replacement policy

As the cache fills up it needs to remove blocks to make room for new ones coming in. In this phase your cache will need to be able to perform a Least Recently Used (LRU) and Pseudo Least Recently Used (PLRU) replacement policy.

Policy	On every hit or fill	Victim on a miss
LRU	The cache is updated, with the new block as the most recently used.	The lowest-numbered invalid way, otherwise the least recently used way in the set.
PLRU	Every tree node on the path to the block points away from it.	The lowest-numbered invalid way, otherwise the way the tree bits lead to.

4.3.1 LRU

LRU tracks the order in which the ways of a set were used. Whenever a block is used it is then set as the most recently used (MRU) block. Whenever a block needs to be replaced to make room for a new one the victim is determined by whichever block in the set has not been used for the longest amount of time.

Breakdown

One 4-way set, all blocks valid. Each row lists the ways from most recently used (left) to least recently used (right). A hit or fill moves that way to the front, while the victim is always the way at the back.

Event	Order (MRU → LRU)	Victim
Start	0, 1, 2, 3	way 3
Hit on way 2	2, 0, 1, 3	way 3
Hit on way 3	3, 2, 0, 1	way 1
Miss, fill way 1	1, 3, 2, 0	way 0

The miss in the last row evicts way 1, the victim from the row above, and the new block becomes most recently used.

Note

Each set is independent. Within a set, you can think of LRU as a queue of ways. In hardware, is implementing a true register queue the best way to do it? Consider the area or power overhead of shifting registers around on every access. There are other ways to implement LRU that are more efficient.

4.3.2 Pseudo-LRU (PLRU)

Figure 1 shows a Pseudo-LRU (PLRU) tree for a 4-way set. PLRU is a tree-based approximate LRU. It is used to reduce the overhead of tracking the exact order of every way within a set. The tree is structured as a binary tree, where the bit determines the direction to the least recently used way. As ways within a set are accessed, the bits used to traverse the tree are flipped to point away from the most recently accessed way. On a miss, the tree is traversed **without updating** to find the way that will be evicted.

4.4 Write policy

Once a cache holds a copy of a block, a store creates two copies of the same data, one in the cache and one in main memory. A write policy decides when main memory gets updated. In this phase,

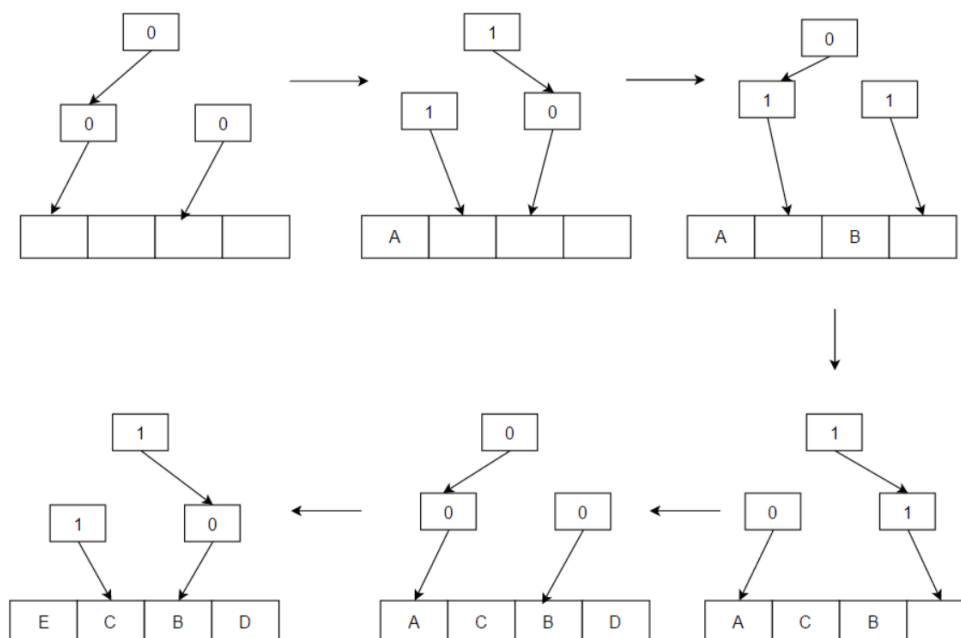


Figure 1: A PLRU tree for a 4-way set.

stores that hit are **write-back** and stores that miss are **write-through**.

Access	Behavior
Store hit	Write-back
Store miss	Write-through, no allocate

4.4.1 Write-back

A write-back cache only updates the block in the cache on a store and marks the block as **dirty**. Main memory is updated later, when the dirty block is evicted, by writing the whole block back.

4.4.2 Write-through

A write-through cache sends every store straight to main memory, so memory always holds the latest data. In this phase it is used only for store misses, and with **no allocate**: only the stored bytes are written to memory, and the block is not brought into the cache.

4.5 Associativity

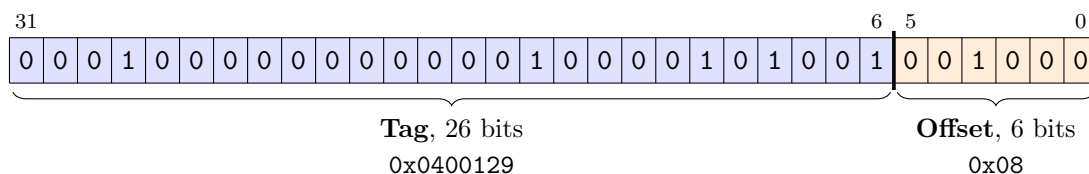
Associativity defines how the cache and memory are grouped into *sets*.

4.5.1 Fully associative

Early caches considered every line within the cache as a part of a single *set*, meaning that the replacement policy would be applied to the entire cache. This is known as *fully associative*. However, there are always tradeoffs. A single set cache is greedy, as it always evicts the least recently used

line in the entire cache. It additionally has area, energy, and performance overheads from having to check the entire cache on access and consider every line as a part of the replacement policy.

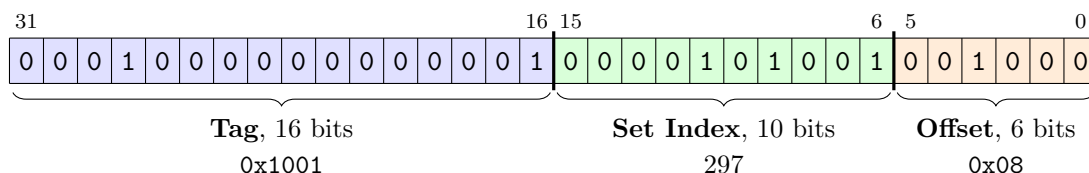
For a fully associative cache, every tag is an individual block address. The lower bits of the address are the block index, which is used to select the word within the block. For a 16 KB fully associative cache with 64-byte blocks (256 ways, 1 set), the same address splits as follows.



4.5.2 Direct mapped

The opposite of this is *direct mapped*. In a direct mapped cache, each line is assigned a different memory region, which is defined as a *set*. Any time a new line is loaded from memory, it replaces the line in the set that corresponds to its memory region. This is a very simple and efficient design, requiring no extra logic for replacement policies, but is conversely inefficient as there is no way to algorithmically identify **what** should be kept in the cache and **what** should be evicted.

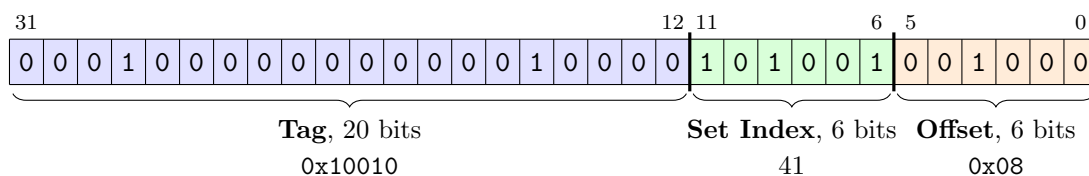
In a direct mapped cache, each line is assigned to a specific set. The lower bits of the address are the block index, which is used to select the word within the block. The next bits are the set index, which is used to select the set that the line belongs to. For a 64 KB direct mapped cache with 64-byte blocks (1024 sets), the same address splits as follows.



4.5.3 Set associative

A middle ground between these two extremes is *set associative*. In a set associative cache, lines are grouped into sets, and each set holds a number of lines. Each memory address is assigned one set that it is allowed to be placed in. The set is determined by the index bit of the address, but it can go in any way of that set. This allows for more flexibility in what is stored in the cache, while still maintaining some structure to reduce overhead.

With more ways per set there are fewer sets, so the index shrinks and the tag grows. For a 16 KB, 4-way cache with 64-byte blocks (64 sets), the same address splits as follows.



4.5.4 Legal configurations

Parameter	Default	Meaning
EVICT_POLICY	0	Replacement policy: 0 = LRU, 1 = PLRU.
NUM_WAYS	1	Blocks per set. 1 = direct mapped; equal to the number of blocks = fully associative.
CACHE_SIZE	16	Data capacity in KB. Tags, valid and dirty bits are not counted.
BLOCK_SIZE	128	Block size in bytes.
PREFETCH_EN	0	Enable next line prefetch: 1 = on, 0 = off.

Listing 1: The cache module header

```

module cache #(
    parameter RESET_ADDR = 32'h00400000,
    parameter EVICT_POLICY = 0,
    parameter NUM_WAYS = 1,
    parameter CACHE_SIZE = 16,
    parameter BLOCK_SIZE = 128,
    parameter PREFETCH_EN = 0
) (
    /* ... */
);

```

The table above lists the parameters of the cache module, and Listing 1 shows the module header. These parameters are the knobs used to configure the cache. Your cache must work for every configuration that follows the rules in the table below. See Section 2 for how each configuration is graded. If you leave out some of the specification, you can still receive partial credit for the configurations you do implement.

Parameter	Legal values
BLOCK_SIZE	A power of 2, with a minimum size of 4.
NUM_WAYS	Any whole number that divides the number of blocks (e.g. 3, 6 or 12 ways).
Number of sets	Must come out a power of 2, since the index is taken straight from address bits.
CACHE_SIZE	Whole KB. Does not need to be a power of 2 (e.g. 12 KB, 3-way, 64 B blocks = 64 sets).
EVICT_POLICY	0: LRU; 1: PLRU. At least one must be implemented.
PREFETCH_EN	0: off; 1: on. Can be left unimplemented.

The cache stores lines of BLOCK_SIZE bytes. Lines are grouped into sets, and each set holds NUM_WAYS lines. Each memory address is assigned one set that it is allowed to be placed in. The set is determined by the index bit of the address, but it can go in any way of that set. Each line contains a tag, data, and dirty bit alongside its data.

Both hart and cache take the cache parameters below. hart passes them down to the cache it instantiates. The testbench sets them, so use these names exactly.

4.6 Main memory

Your design should assume a working memory, that you can simulate in your testing harness that you design. For both instruction and data memory, you will only need to use 16K 32-bit words (64 KB). The memory is byte addressable, so the addresses will be 32 bits wide. The memory is block aligned, meaning that the lower bits of the address are used to select the word within the block.

Your memory will need to support reads and writes, with a round-trip latency of n cycles (arbitrary). The pipeline of prefetching is implemented in both **your cache** and **the memory**. Your cache cannot output two accesses in one cycle, and thus will need a queue to hold accesses. Your memory will also need to support pipelined accesses, so that it can accept a new one every cycle. The memory will return the data in order, so you will need to track which access is being returned.

Item	Spec
Location	In the testbench, outside your design.
Size	Set by the testbench (default 64 KB). Your design must not depend on it.
Width	One whole block per transfer (BLOCK.SIZE bytes).
Read latency	n cycles from the request to the data. Wait for <code>i_mem_valid</code> ; do not count cycles.
Reads	Pipelined: a new read can be requested every cycle. Answers come back in order.
Writes	Applied on the clock edge they are sent, with byte enables. No stall.

Port	Dir/Width	Meaning
<code>o_mem_rd</code>	out	Block read request. High for one cycle per request.
<code>o_mem_rd_addr</code>	out [31:0]	Block-aligned address of the block to read.
<code>i_mem_valid</code>	in	High for one cycle when a requested block arrives.
<code>i_mem_rd_data</code>	in [BLOCK.SIZE*8-1:0]	The whole block, valid while <code>i_mem_valid</code> is high.
<code>o_mem_wr</code>	out	Block write. High for one cycle per write.
<code>o_mem_wr_addr</code>	out [31:0]	Block-aligned address of the write.
<code>o_mem_wr_data</code>	out [BLOCK.SIZE*8-1:0]	Write data, in place within the block.
<code>o_mem_wr_be</code>	out [BLOCK.SIZE-1:0]	One bit per byte. All ones for a dirty write-back; only the stored bytes for a write-through.

4.7 Stalling on a miss

When a load misses, the load stays in the MEM stage until its line arrives. Every pipeline register holds its value while the cache is waiting, and no flush may happen until the stall ends. The example below uses a memory that returns the block 4 cycles after the request, which is the delay of the base configuration; your design must work for any delay.

Cycle	Cache / memory	CPU
1	Load misses; block read sent to memory	Load in MEM, stalled
2	Memory working	Stalled
3	Memory working	Stalled
4	Memory working	Stalled
5	Block arrives; stored in the cache, data to the CPU	Load finishes
6	Ready for the next access	Next instruction's memory access

4.8 Next-line prefetch

Programs often go through memory in order, so when there is a miss on one line we can assume that the one next to it is likely also going to be needed soon. With next line prefetching, whenever there is a load miss, the line directly after it is also fetched. This way, we can avoid a potential miss in the future.

Rule	Detail
Address	The missed line's address + <code>BLOCK_SIZE</code> , i.e. <code>{tag, index} + 1</code> . An index overflow carries into the tag.
Trigger	Only a load miss. Hits never prefetch, and a write-through store miss never prefetches.
Skipped	If the next line is already in the cache.
Request	Sent the cycle after the demand request. Both reads are in flight at once.
CPU	Resumes when the demand line arrives. It does not wait for the prefetch.
Placement	Victim chosen when the prefetched line arrives; a dirty victim is written back. The line is stored clean, as most recently used.

With the same 4-cycle memory, the timeline is:

Cycle	Cache / memory	CPU
1	Load misses; demand read sent	Load in MEM, stalled
2	Next-line read sent	Stalled
3–4	Memory working on both reads	Stalled
5	Demand line arrives; stored, data to the CPU	Load finishes
6	Prefetched line arrives and is stored	Running

4.8.1 Prefetch collisions

When a prefetched line returns from memory, it updates the cache while the cpu is active. If the CPU is also accessing the cache at the same time, and happens to be accessing the same set, then there is a collision. There are two ways to handle this.

- Option A: Both cache updates in one cycle. This allows the CPU to continue without stalling, but adds area and power overhead, along with logic complexity.
- Option B: The CPU is stalled for one cycle while the prefetch updates the cache, then the CPU access happens the next cycle. This is simpler to implement, but adds a stall cycle.

In both cases, the prefetch line **has priority** over the CPU access. This means that it updates the cache first.

4.9 Memory regions

Instructions still start at 0x00400000 and data at 0x10010000, as in Phase 4. The cache works on full 32-bit addresses, so data addresses need no special handling: the high bits simply become part of the tag.

4.10 Retire interface

These outputs are not part of RV32I. They exist for the testbench to validate your design and its functionality. Set each signal appropriately every cycle. As in Phase 4, `o_retire_valid` is not guaranteed high every cycle. Cache stalls leave it low for as long as the pipeline is frozen, and that is not a defect.

Port	Width	Meaning
<code>o_retire_valid</code>	1	An instruction retired this cycle.
<code>o_retire_inst</code>	[31:0]	The raw fetched instruction word.
<code>o_retire_trap</code>	1	Illegal encoding, or a misaligned access.
<code>o_retire_halt</code>	1	The instruction is ebreak .
<code>o_retire_rs1_raddr</code>	[4:0]	Source register 1, 5'd0 if not read .
<code>o_retire_rs1_rdata</code>	[31:0]	The architecturally correct value rs1 used, reflecting any forwarding.
<code>o_retire_rs2_raddr</code>	[4:0]	Source register 2, 5'd0 if not read .
<code>o_retire_rs2_rdata</code>	[31:0]	The architecturally correct value rs2 used, reflecting any forwarding.
<code>o_retire_rd_waddr</code>	[4:0]	Destination register, 5'd0 if no register is written .
<code>o_retire_rd_wdata</code>	[31:0]	Value written to rd .
<code>o_retire_pc</code>	[31:0]	PC this instruction was fetched from.
<code>o_retire_next_pc</code>	[31:0]	PC + 4, or the branch or jump target when taken.
The retiring instruction's own data memory access, as presented to the data cache, timed to the writeback cycle:		
<code>o_retire_dmem_addr</code>	[31:0]	Word-aligned address of the load or store.
<code>o_retire_dmem_ren</code>	1	The instruction was a load. Never high with <code>o_retire_dmem_wen</code> .
<code>o_retire_dmem_wen</code>	1	The instruction was a store.
<code>o_retire_dmem_mask</code>	[3:0]	One bit per byte lane of the word, as in Phase 4.
<code>o_retire_dmem_wdata</code>	[31:0]	Store data, in place within the word. Only the masked lanes matter.
<code>o_retire_dmem_rdata</code>	[31:0]	The full word the data cache returned for a load.

Note

The testbench uses `o_retire_valid` and `o_retire_halt` to detect the final **ebreak**. Every port in this table must still exist with these exact names and widths or your hart will not compile against the testbench, and keep every value correct as in Phase 4: the retire trace is how you find the first wrong instruction when the memory check fails.

4.10.1 Traps

Traps work exactly as in Phase 4. A trapped instruction retires with `o_retire_trap` set, execution continues at pc+4, and the instruction has no side effects. For a misaligned load or store this now means it must not reach the cache at all, meaning no lookup, no miss, no line read and no write to main memory.

4.11 Design requirements

The tables below sum up what your design must do and which choices are yours.

Required	Detail
Parameters	EVICT_POLICY, NUM_WAYS, CACHE_SIZE, BLOCK_SIZE, PREFETCH_EN, with these exact names. The testbench sets them.
Base config	LRU, fully associative, 16 KB, 128 B blocks, no prefetch. This is the bare minimum implementation to receive any partial credit .
Additional Configurations	Every legal configuration in Section 4.5.4.
Replacement	LRU and PLRU. An invalid way is always used before evicting a valid one.
Write policy	Write-back on a hit, write-through (no allocate) on a miss.
Memory interface	The ports in Section 4.6. Read requests are one-cycle pulses.
Waiting on memory	Wait for <code>i_mem_valid</code> . Main memory's size and latency are set by the testbench. This means your design must work for an arbitrary memory delay.
Prefetch	Next line only, only on a load miss, skipped if the line is already cached. This is implemented as a queue to memory, with only one request sent to memory each cycle.

Left to you	Detail
Cache control	Your state machine and how the cache tells the pipeline to stall.
Replacement state	How you store the LRU order and the PLRU tree bits.
Cache storage	How you lay out tags, valid bits, dirty bits and data.
Next-line prefetch	How you implement the queue to memory.
Prefetch collisions	How to handle a load/store in the cycle a prefetched line arrives (Section 4.8.1).

5 Testing Your Work

You must write your own testbenches. Two trace files are provided for one test, in the `traces/vectors/` directory.

Instruction memory file	Final data memory file
<code>random_program.hex</code>	<code>random_dmem.hex</code>

5.1 Trace files

`random_program.hex` is the initial instruction memory. It has one 32-bit word per line, the first word at address `0x00400000`. It holds about 12000 random straight-line RV32I instructions whose loads and stores cover 64 KB of data starting at `0x10010000`, ending with an `ebreak`. Initialize your data memory to all 0's.

`random_dmem.hex` is the final data memory. It has 16384 words, one per line, starting at address `0x10010000`.

6 Learnings Overview

These are the learnings you will gain from this phase:

- **Caches.** What a cache is and how it is organized. Blocks, lines, tags, valid bits and dirty bits, and how an address splits into tag, index and offset.
- **Associativity.** Direct mapped, set associative and fully associative caches, and the trade-off between hit rate and hardware cost.
- **Replacement policies.** LRU and PLRU, how each tracks use, how each picks a victim, and the trade-off between accuracy and hardware cost.
- **Write policies.** Write-back with dirty bits on a store hit, write-through with no allocate on a store miss.
- **Next-line prefetching.** Fetching the following line, $(\{\text{tag_address}, \text{set_address}\} + 1)$, on a miss, when to skip it, and handling a prefetched line that arrives while the CPU is using the cache.
- **Main memory interface.** Block-wide reads and writes, pipelined requests, and overlapped memory response latency. Your cache should wait for a valid signal of counting cycles.
- **Cache integration.** Stalling a pipelined CPU on a miss by freezing its pipeline registers and requesting accesses from the cache that intermediates between the CPU and main memory.
- **Parameterized design.** One cache module that works for every legal configuration of size, block size, ways, replacement policy and prefetch.